iacc instituto profesional

EVALUACIÓN

Herramientas de QA
Semana 5

Amparo Valdivia
21, julio de 2025
Técnico en Análisis y Programación.

DESARROLLO:

Para soluciones de banca en línea y aplicaciones móviles de clientes, los softwares que entregan estos servicios para cumplir con los estándares de calidad generalmente realizan pruebas en una fase inicial funcionales de manera manual, lo que significa invertir tiempo y diversos recursos humanos, para agilizar el proceso, algunas empresas paulatinamente han incorporado metodologías ágiles a estos entornos.

Debido a que las pruebas manuales, no solo requieren especialmente experiencia y tiempo, ocurren retrasos en el cumplimiento de metas, en otras ocasiones la poca rigurosidad en la documentación de los pasos que se han seguido en la solución del problema impide la regresión de los procesos al no quedar grabados, optimizar su proceso de pruebas mediante la automatización de las mismas, es un recurso que a continuación exploraré.

SELENIUM	CYPRESS	TESTCOMPLETE
La historia comienza en 2004 en ThoughtWorks, Chicago, con Jason Huggins desarrollando el modo Core como "JavaScriptTestRunner" para las pruebas de una aplicación interna de Tiempo y Gastos (Python, Plone). Shinya Kasatani, en Japón, se interesó	**La historia** Nació como una reacción a SELENIUM, desde sus inicios se reconoció como un gran aporte empresarial y la importancia de automatizar pruebas, es así que, A lo largo de 2019 y 2020, a medida que el desarrollo ágil se convertía en la estrategia por defecto de muchas empresas, sus ingresos se	**La historia** **TestComplete** fue originalmente desarrollado por **AutomatedQA**. Antes de la fusión, de SmartBear y AutomatedQA ya era conocida por sus herramientas de automatización de pruebas. Las primeras versiones de TestComplete se lanzaron a

en Selenium y se dio cuenta de que podía integrar el código principal en un módulo IDE para el navegador Firefox, además de grabar y reproducir pruebas en el mismo complemento. Esta herramienta resultó ser una revelación en más sentidos de lo que se había pensado inicialmente,	multiplicaron por diez, su fundador dijo Sr. Mann. “Pero, sinceramente, después de tanto trabajo, solo veo cuánto trabajo se necesita para establecer un equilibrio entre la codificación y las pruebas. Las pruebas son, y siempre serán, la parte más importante del desarrollo”	principios de los años 2000 (aproximadamente 2002-2003). En ese momento, se destacaba por ser una de las pocas herramientas que ofrecía una interfaz gráfica de usuario (GUI) para la creación de pruebas automatizadas, lo que la hacía más accesible para usuarios sin profundos conocimientos de programación, creo que este es el motivo de que actualmente muchas empresas como CISCO la utilicen.
Instalación, Descargar para Firefox desde su página oficial, https://www.selenium.dev/selenium-ide/, opción Firefox, descomprimir, guardar como archivo.	**Instalación** Descargar desde la página oficial, https://www.cypress.io/, antes de instalar Cypress, asegúrese de tener el archivo package.json en la raíz de su proyecto. Si necesita crearlo, puede ejecutar el comando init de su gestor de paquetes. Cypress Para su uso require Node.js in order to install.	**Instalación** Al buscar Test complete y alguna página oficial parecería que no tuviera, pero se reconoce a que es un producto de una empresa de software llamada **SmartBear**. SmartBear desarrolla varias herramientas populares para desarrolladores y equipos de QA, y

	We support the versions listed below: • **Node.js** 18.x, 20.x, 22.x and above Para descarga directa, no es posible grabar ejecuciones en Cypress Cloud desde la descarga directa. Esta descarga solo es una forma rápida de probar Cypress. Para grabar pruebas en Cypress Cloud, deberá instalar Cypress como una dependencia del paquete, guardar como archivo y antes descomprimir.	TestComplete es una de sus soluciones estrella. Para buscar su descarga ingresar a https://smartbear.com/product/testcomplete/,
Costo, De código abierto, gratuito.	**Costo,** De código abierto, gratuito.	**Costo,** Posee una Prueba gratuita de 14 días No se requiere tarjeta de crédito, siendo una plataforma de pago, después de esta fecha.
Lenguajes, Se pueden probar aplicaciones escritas en Java, Python, C#, Ruby, JavaScript, Kotlin	**Lenguajes,** Source (JavaScript), por eso necesita Node de java Script.	**Lenguajes,**JavaScript, VBScript, Python, C#, DelphiScript

Características, Framework, con gran Librería, para realizar pruebas, grabarlas y reproducirlas entre otras funciones, de usabilidad amigable con el usuario. -Web (UI), Mobile (con Appium). -WebDriver (interactúa con el navegador externamente) -Automatiza interacciones con interfaces de usuario web, simulando el comportamiento de un usuario real.	**Características,** Con Cypress, puede crear fácilmente pruebas para sus aplicaciones web modernas, depurarlas visualmente y ejecutarlas automáticamente en sus compilaciones de integración continua, para probar Web (UI), Componentes, entrega capturas y videos, su plataforma tiene un buen soporte al usuario es clara y muestra exitosamente sus contenidos, por ejemplo, **What you'll learn** • How to write your first end-to-end test in Cypress using an example app • How to use Cypress commands to interact with elements on a page • How to make assertions about the state of the application.	**Características** TestComplete reconoce objetos y controles en aplicaciones probadas y ofrece comandos especiales para simular las acciones de los usuarios con ellos. También ofrece puntos de control específicos que le permiten verificar fácilmente el estado de la aplicación durante la ejecución de la prueba. Web (UI), Desktop, Mobile (nativa/web), su slogan es Realice pruebas funcionales automatizadas con una herramienta diseñada para cualquier nivel de experiencia. Con TestComplete, cree, ejecute y mantenga fácilmente pruebas de IU. Agilice las pruebas a escala. Crea scripts con lenguajes modernos o pruebas sin código Accede a la nube de dispositivos para realizar pruebas en navegadores y dispositivos móviles

		Integración inmediata con las herramientas de tu ecosistema Su framework se ve amigable, con iconos claros y muchos colores, su barra de tareas es similar a otras.

Como argumenta Cypress en su página web, sobre su historia en letras muy grandes es que **Las pruebas son la clave para la innovación continua, este hecho nos demuestra que el aprendizaje se sitúa justo después de la prueba error-> documentación -> regresión->nuevo registro->prueba otra vez->éxito seguro->aprendizaje continuo->>innovación.**

Fortalezas de CELENIUM IDE

Primero es necesario aportar con mi experiencia ya que Selenium IDE es conocido por Baja Barrera de Entrada y Facilidad de Uso, cierto, una vez instalado su framework es muy intuitivo, permite grabar una secuencia de acciones de una aplicación que se proporcione y reproducirla en los mismos pasos como la interacción se presentó.

Si bien es cierto en reiteradas ocasiones se dice que es muy útil para personas que no sepan codificar, en mi caso a medida que he ido aprendiendo a hacerlo, cuando uno diseña un logaritmo y comienza a codificar realiza pruebas manuales en el localhost de mi computadora, por ejemplo, puedo crear un HTML, un CSS o usar Bootstrap, luego el backend con javascrip, Python, PHP, solo algunas líneas como sería un formulario, o en un MVC que cree una API con algunas vistas donde yo pueda interactuar, sí he realizado

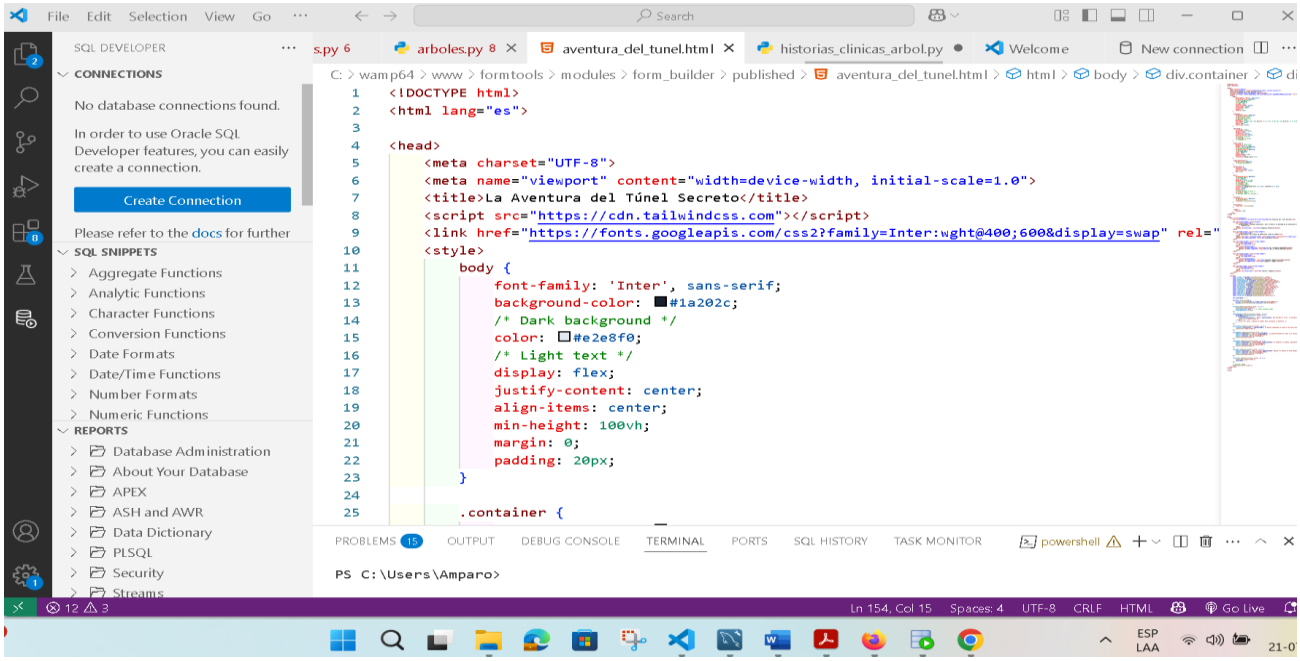
pruebas manuales, generalmente en el localhost aparecen los errores, pero Selenium IDE me pareció una herramienta muy potente, si es mucho más rápido que una prueba manual y creo que tal vez su máximo punto es que se siente y ve muy real, las regresiones y visualizaciones permiten pensar en mejoras inmediatas, es excelente para la planificación de mejoras, es muy profesional y directamente práctico, creo que ligado a la codificación con metodologías ágiles para crear y probar trozos de código, como sistema automatizado de pruebas a la altura de API, es un complemento que permite a los desarrolladores compartir visualizaciones, entornos de prueba y crear conexiones de trabajo mucho más profundas con otros profesionales, solo puedo observar con su unión una entrega de proyecto satisfactorio y de gran calidad para el cliente durante el desarrollo y las correcciones finales, es algo que se puede mostrar y compartir para discutir el proyecto muy didácticamente.

Prueba de código y API interfaz de usuario (UI) con Selenium IDE.

Para descargar Selenium IDE, realicé la descarga de Firefox en mi pc,

Luego en la APP Store busqué Selenium IDE y comencé su descarga, con un archivo zip, el cual se descomprimió y procedió a guardar sus extensiones en un archivo.

Imagen Visual Studio Code con el código de la API mi Pc.



La Api es de la aventura del túnel y es la que voy a probar, la **interfaz de usuario (UI)**

```
<!DOCTYPE html>
<html lang="es">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>La Aventura del Túnel Secreto</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&display=swa
p" rel="stylesheet">
    <style>
        body {
            font-family: 'Inter', sans-serif;
            background-color: #1a202c;
            /* Dark background */
            color: #e2e8f0;
            /* Light text */
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
            margin: 0;
            padding: 20px;
```

```
        }

        .container {
            background-color: #2d3748;
            /* Darker container */
            border-radius: 1rem;
            /* Rounded corners */
            padding: 2.5rem;
            box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -2px
rgba(0, 0, 0, 0.05);
            max-width: 600px;
            width: 100%;
            text-align: center;
        }

        .text-area {
            min-height: 100px;
            margin-bottom: 1.5rem;
            display: flex;
            align-items: center;
            justify-content: center;
            font-size: 1.125rem;
            /* text-lg */
            line-height: 1.75rem;
            /* leading-7 */
        }

        .input-field {
            padding: 0.75rem 1rem;
            border-radius: 0.5rem;
            border: 1px solid #4a5568;
            /* border-gray-600 */
            background-color: #2d3748;
            /* bg-gray-700 */
            color: #e2e8f0;
            width: 100%;
            margin-bottom: 1rem;
            outline: none;
            transition: border-color 0.2s;
        }

        .input-field:focus {
            border-color: #63b3ed;
            /* border-blue-400 */
        }

        .button-group {
```

```
            display: flex;
            flex-wrap: wrap;
            justify-content: center;
            gap: 1rem;
        }

        .btn {
            background-color: #4299e1;
            /* bg-blue-500 */
            color: white;
            padding: 0.75rem 1.5rem;
            border-radius: 0.75rem;
            /* rounded-xl */
            font-weight: 600;
            /* font-semibold */
            transition: background-color 0.3s ease, transform 0.1s ease;
            cursor: pointer;
            border: none;
            flex-grow: 1;
            /* Allow buttons to grow */
            min-width: 120px;
            /* Minimum width for buttons */
        }

        .btn:hover {
            background-color: #3182ce;
            /* bg-blue-600 */
            transform: translateY(-2px);
        }

        .btn:active {
            transform: translateY(0);
        }

        .hidden {
            display: none;
        }
    </style>
</head>

<body>
    <div class="container">
        <h1 class="text-3xl font-bold mb-6 text-blue-300">¡La Aventura del
Túnel Secreto!</h1>

        <div id="start-screen" class="screen">
            <div class="text-area">
```

```
                <p>Bienvenido, valiente aventurero. Estás frente a la entrada
de un misterioso túnel. ¿Te atreves a
                    entrar?</p>
            </div>
            <button id="start-btn" class="btn">Comenzar Aventura</button>
        </div>

        <div id="name-screen" class="screen hidden">
            <div class="text-area">
                <p>¡Excelente! Pero antes de adentrarte, dime tu nombre:</p>
            </div>
            <input type="text" id="player-name" class="input-field"
placeholder="Tu nombre aquí...">
            <button id="submit-name-btn" class="btn">Continuar</button>
        </div>

        <div id="room1-screen" class="screen hidden">
            <div class="text-area">
                <p id="room1-text"></p>
            </div>
            <div class="button-group">
                <button id="door-left-btn" class="btn">Ir por la Puerta
Izquierda</button>
                <button id="door-right-btn" class="btn">Ir por la Puerta
Derecha</button>
            </div>
        </div>

        <div id="room2-screen" class="screen hidden">
            <div class="text-area">
                <p id="room2-text"></p>
            </div>
            <div class="button-group">
                <button id="examine-btn" class="btn">Examinar Objeto
Brillante</button>
                <button id="ignore-btn" class="btn">Ignorar y Seguir</button>
            </div>
        </div>

        <div id="end-screen" class="screen hidden">
            <div class="text-area">
                <p id="end-text"></p>
            </div>
            <button id="restart-btn" class="btn">Volver a Empezar</button>
        </div>
    </div>
```

```
    <script>
        const screens = document.querySelectorAll('.screen');
        const startBtn = document.getElementById('start-btn');
        const nameScreen = document.getElementById('name-screen');
        const playerNameInput = document.getElementById('player-name');
        const submitNameBtn = document.getElementById('submit-name-btn');
        const room1Screen = document.getElementById('room1-screen');
        const room1Text = document.getElementById('room1-text');
        const doorLeftBtn = document.getElementById('door-left-btn');
        const doorRightBtn = document.getElementById('door-right-btn');
        const room2Screen = document.getElementById('room2-screen');
        const room2Text = document.getElementById('room2-text');
        const examineBtn = document.getElementById('examine-btn');
        const ignoreBtn = document.getElementById('ignore-btn');
        const endScreen = document.getElementById('end-screen');
        const endText = document.getElementById('end-text');
        const restartBtn = document.getElementById('restart-btn');

        let playerName = '';

        function showScreen(id) {
            screens.forEach(screen => screen.classList.add('hidden'));
            document.getElementById(id).classList.remove('hidden');
        }

        startBtn.addEventListener('click', () => {
            showScreen('name-screen');
            playerNameInput.value = ''; // Clear previous input
            playerNameInput.focus();
        });

        submitNameBtn.addEventListener('click', () => {
            playerName = playerNameInput.value.trim();
            if (playerName) {
                room1Text.textContent = `¡Hola, ${playerName}! Has entrado al
túnel. Te encuentras en una encrucijada con dos puertas. ¿Cuál eliges?`;
                showScreen('room1-screen');
            } else {
                alert('Por favor, ingresa tu nombre para continuar la
aventura.');
            }
        });

        doorLeftBtn.addEventListener('click', () => {
            room2Text.textContent = `${playerName}, la puerta izquierda te
lleva a una sala oscura con un objeto brillante en el centro. ¿Qué haces?`;
            showScreen('room2-screen');
```

```
        });

        doorRightBtn.addEventListener('click', () => {
            endText.textContent = `¡Oh no, ${playerName}! La puerta derecha
te llevó a un callejón sin salida. ¡Aventura terminada!`;
            endText.classList.remove('text-green-400');
            endText.classList.add('text-red-400');
            showScreen('end-screen');
        });

        examineBtn.addEventListener('click', () => {
            endText.textContent = `¡Felicidades, ${playerName}! Al examinar
el objeto, descubres un mapa del tesoro y completas la aventura con éxito.
¡Eres un héroe!`;
            endText.classList.remove('text-red-400');
            endText.classList.add('text-green-400');
            showScreen('end-screen');
        });

        ignoreBtn.addEventListener('click', () => {
            endText.textContent = `¡Qué lástima, ${playerName}! Ignorar el
objeto te hizo perder el camino. La aventura termina aquí sin gloria.`;
            endText.classList.remove('text-green-400');
            endText.classList.add('text-red-400');
            showScreen('end-screen');
        });

        restartBtn.addEventListener('click', () => {
            showScreen('start-screen');
            playerName = '';
        });

        // Initial screen
        showScreen('start-screen');
    </script>
</body>

</html>
```

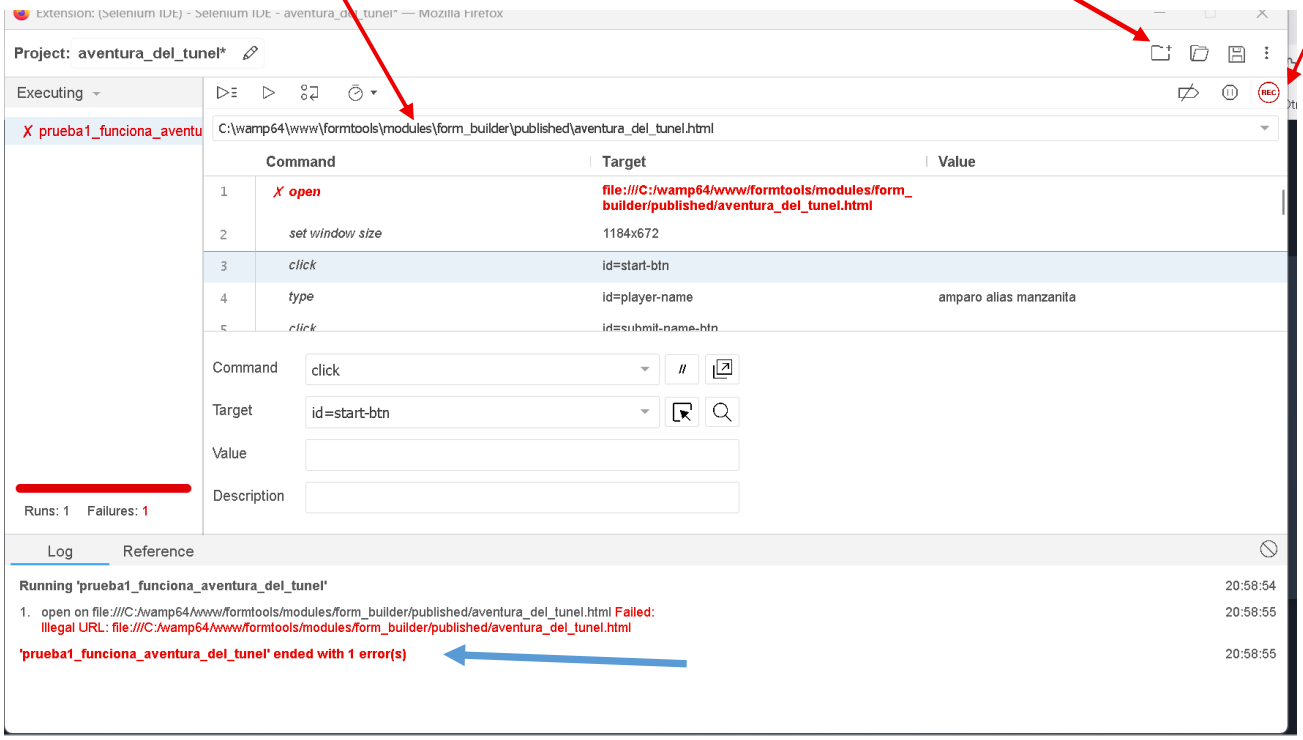
Pantalla de Firefox,

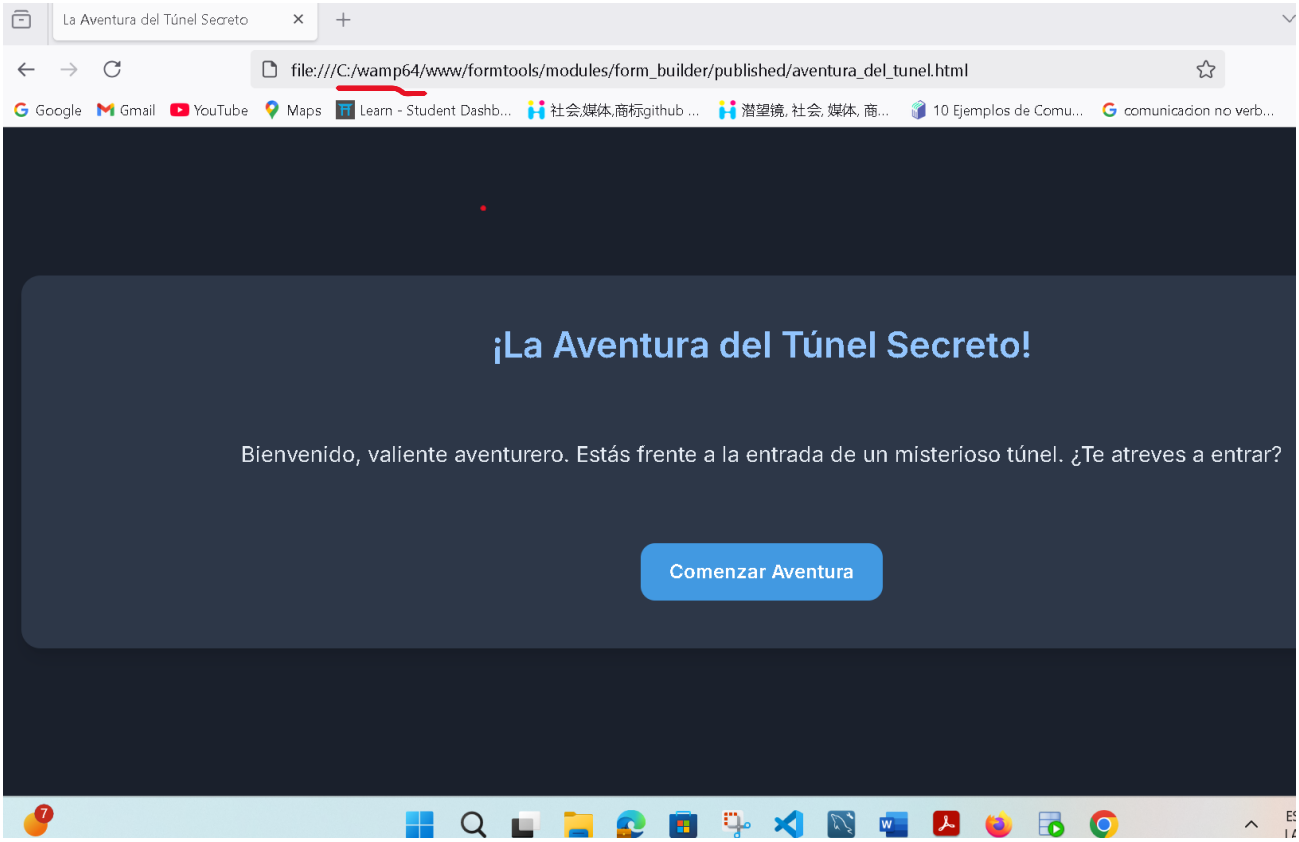
Selenium IDE, clickear y abrir entorno

Primera prueba, ingresar url

Agregar archivo

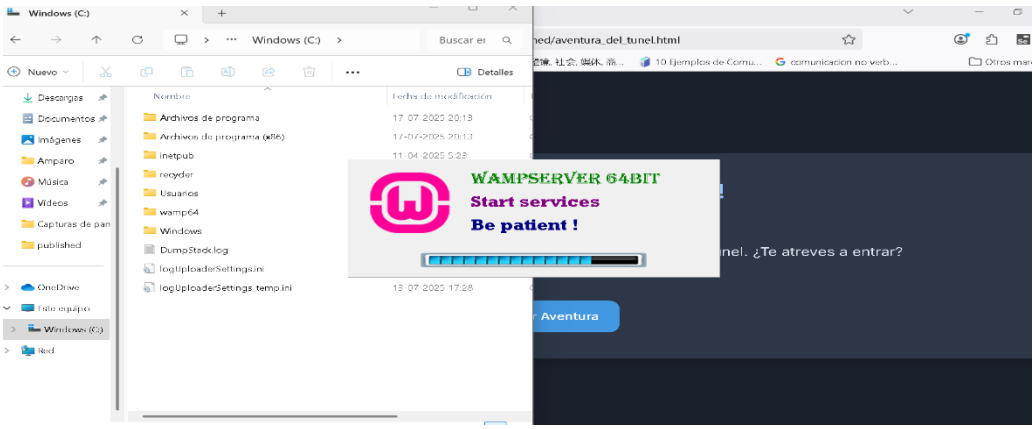
Grabar



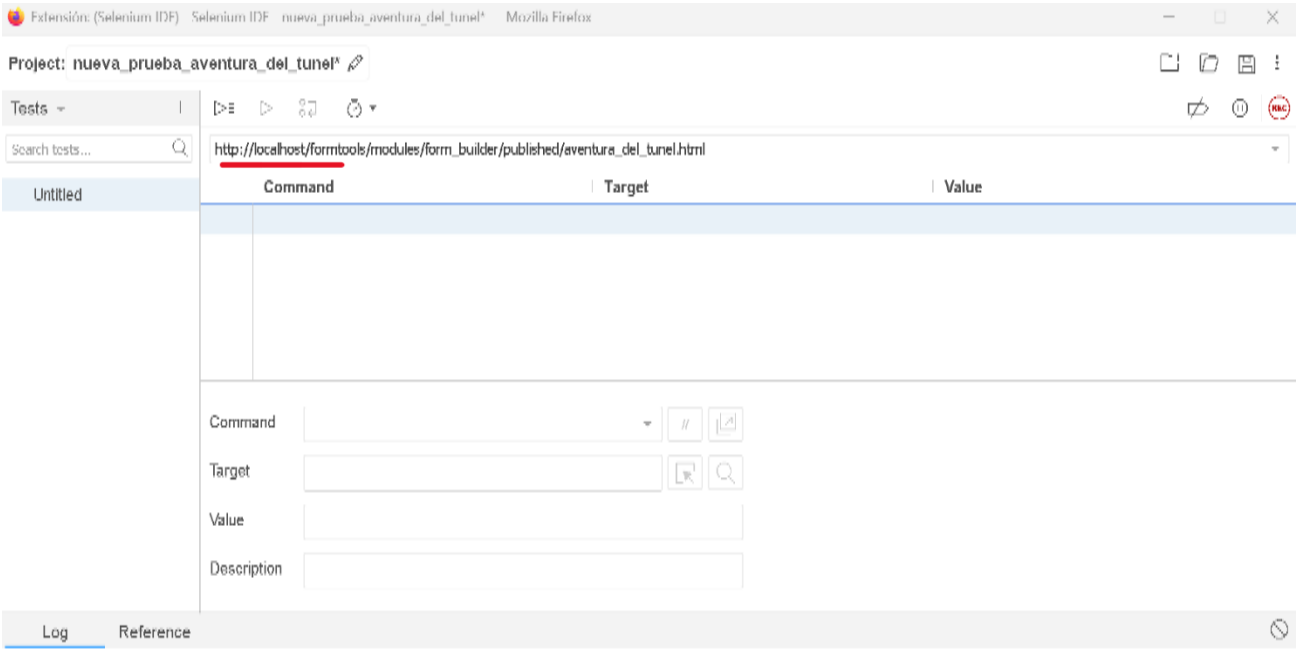


Aunque se grabo la sesión, se presento un error debido a que no abre el servidor Wamp, y la URL no estaba en localhost, este es el escenario que permite registrar los errores.

Abriendo Wamp,



Comienzo de nueva prueba, cambio de wamp… a //localhost…



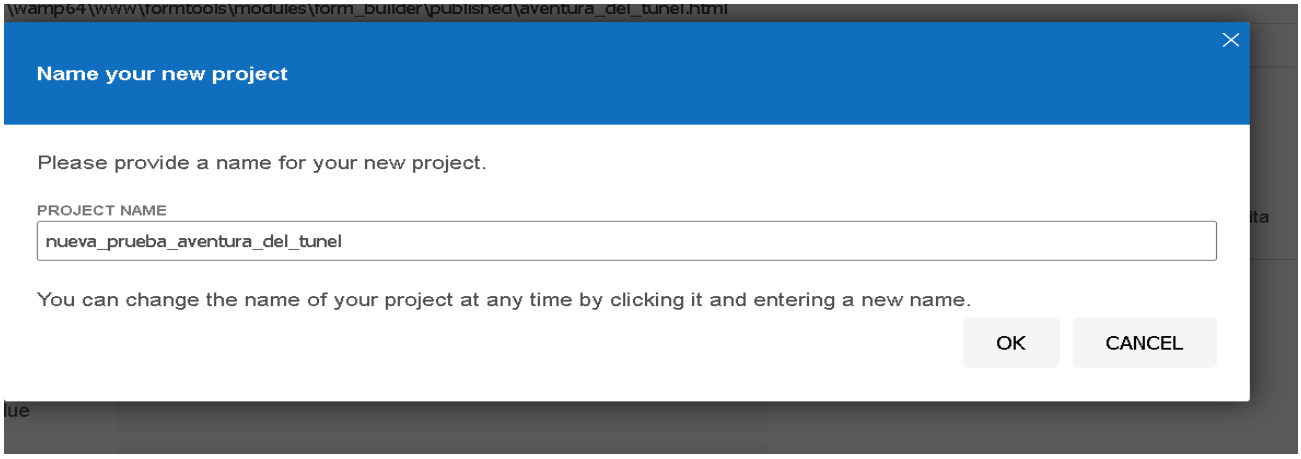
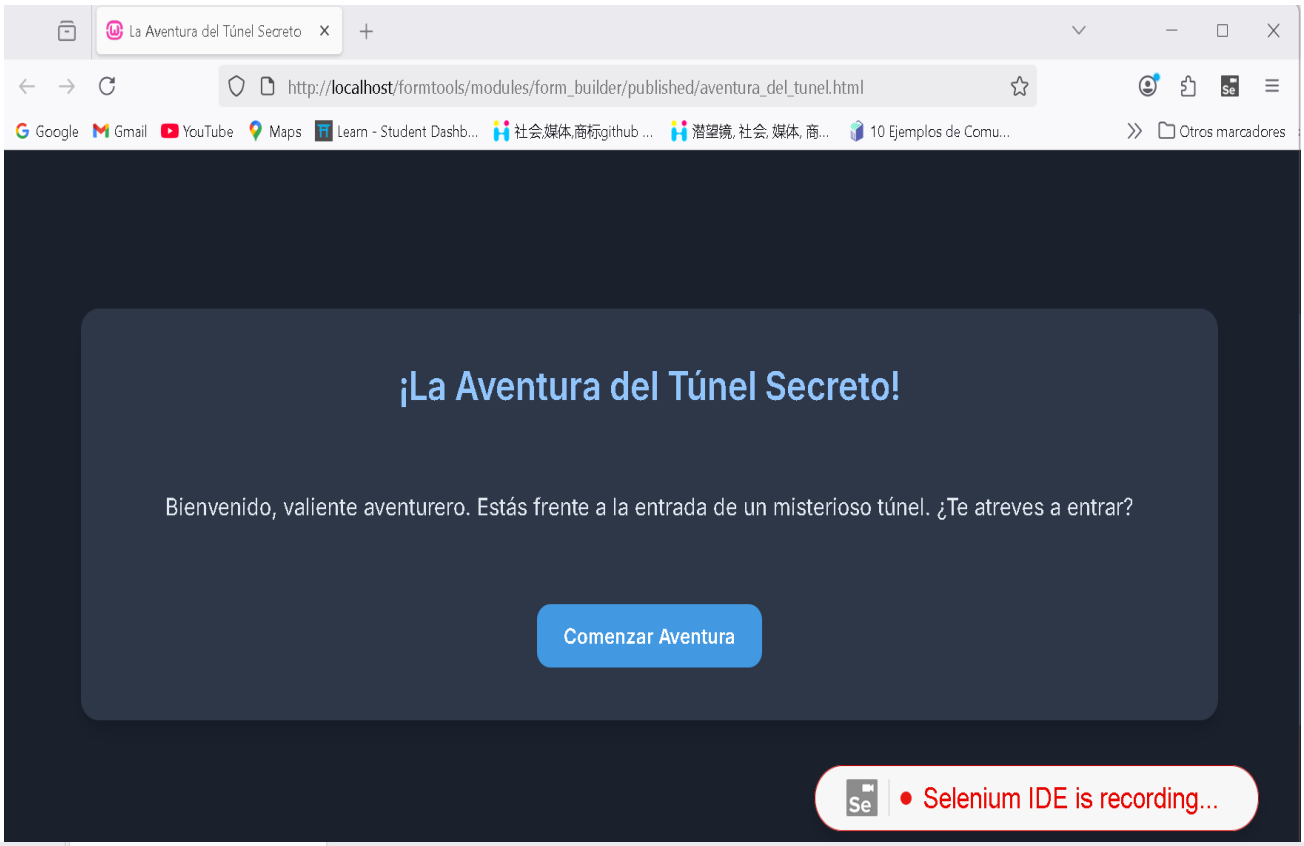
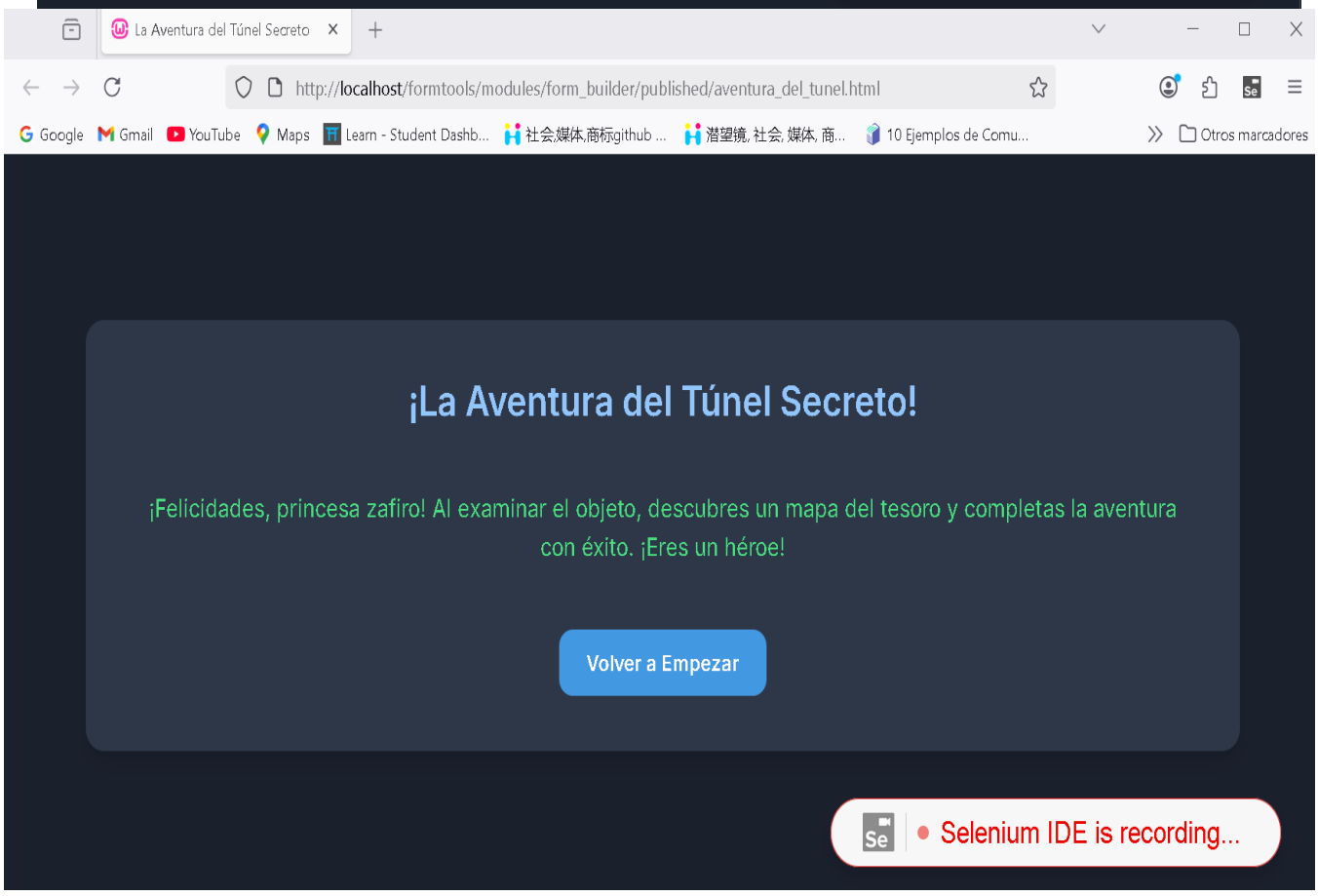
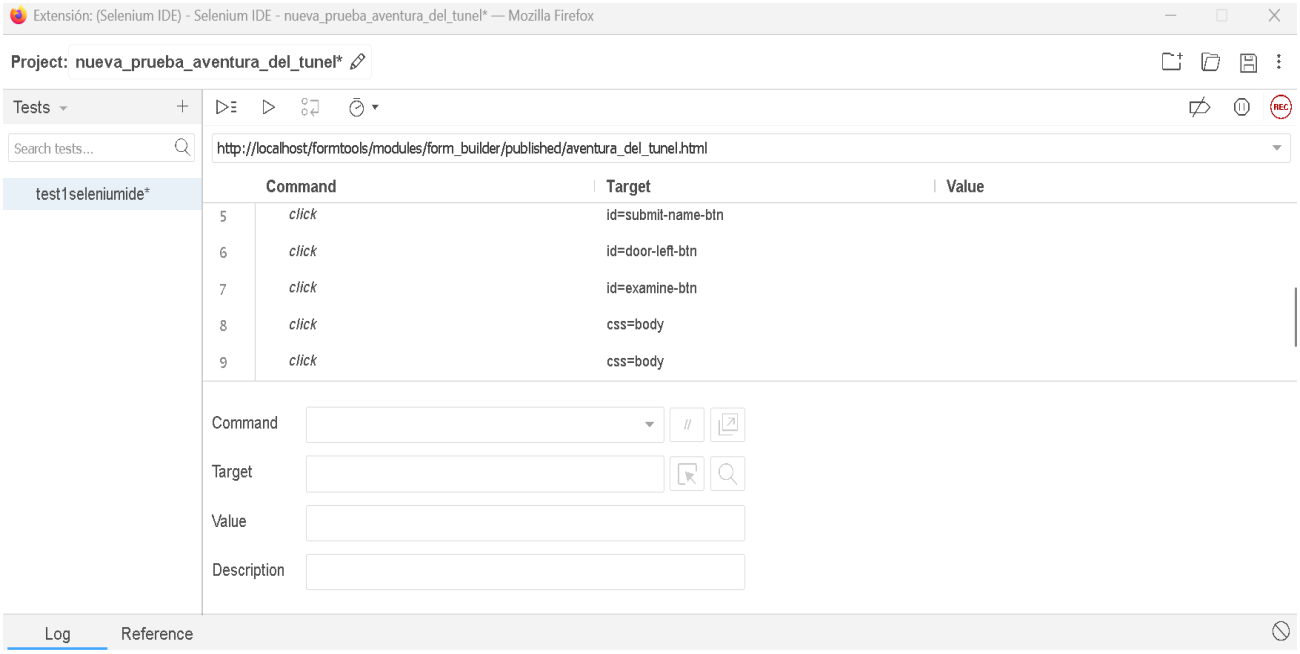


Imagen de inicio grabación aventura en el túnel,





✓ Dejar de grabar y esta vez sin errores, Selenium IDE muestra registro.



Para complementar, después de descubrir Selenium IDE:

- En el sector financiero, es vital para que las pruebas iniciales de nuevas funcionalidades o cambios regulatorios se realicen sea de forma ágil y visual, detectando errores de interfaz o de flujo en etapas muy tempranas.
- Reproducir las pruebas directamente en el navegador visible, en aplicaciones financieras, donde la precisión de la interfaz de usuario y la experiencia del cliente son importantes, Selenium IDE es una gran herramienta.
- Los errores de comportamiento como por ejemplo descuadre imagen, superposición recuadros pueden verse de forma muy real.

- Al ser probada la aplicación por cualquier persona que no sea el desarrollador permite ver la interacción en tiempo real.
- Se puede comprobar la rapidez con que responde la aplicación o si esta tiene botones que no responden.
- Permite verificar rápidamente que las funcionalidades existentes, como transferencias, pagos, visualización de saldos, que son transacciones sensibles.
- Permite ver que las solicitudes como get funcionen.
- Mejora el proceso de reducción de riesgos funcionales.
- Queda clara evidencia de las pruebas.
- La seguridad de realizar pruebas con Selenium IDE es muy importante para determinar exhaustivas pruebas funcionales que el software cumple con los requerimientos necesarios, además de aportar con la documentación que puede servir de respaldo, por ejemplo, en solicitudes, buen funcionamiento de claves, y Validación de Formularios HTML, valida correctamente los datos de entrada, para garantizar la seguridad y la integridad de tus datos.
- Las pruebas automatizadas eliminan el factor de error humano inherente a las pruebas manuales repetitivas.
- Permite ejecutar un gran número de pruebas de regresión de forma consistente y rápida.
- Al usar Selenium IDE, se debe evitar grabar credenciales reales o datos sensibles de clientes. Se deben usar datos de prueba ficticios y seguros.
- Cuando se exporta un test de Selenium IDE, el software (Selenium IDE) no tiene una forma inherente de "obtener" los datos que se usaron durante la grabación en el script exportado.

- "realizar pruebas funcionales para guardar y que visualmente no aparezcan los datos del cliente donde no deban" es una manifestación directa de la importancia del cumplimiento normativo en el testing.
- **Privacidad de Datos:** Asegurar que los datos personales (nombres, direcciones, números de cuenta, saldos) solo sean accesibles por personas autorizadas y en los contextos adecuados.
- Prevención de la exposición de datos sensibles en la interfaz de usuario (UI) o prevención de fugas visuales de datos.
- Herramientas como Selenium IDE son excelentes para este tipo de pruebas porque operan a nivel de UI. Al poder grabar un flujo, y luego usar comandos de aserción (assertText, verifyTextPresent, assertElementNotPresent) para:
- Confirmar que un texto específico (enmascarado) está presente.
- Verificar que un texto sensible (no enmascarado) NO está presente.
- Asegurar que ciertos elementos de la interfaz como botones para ver datos sensibles, no estén visibles para roles no autorizados.

REFERENCIAS BIBLIOGRÁFICAS

https://www.selenium.dev/about/

https://www.selenium.dev/selenium-ide/

https://www.cypress.io/

https://smartbear.com/product/testcomplete/

https://rise.articulate.com/share/26nrkBAnsHKiMLjWsXRR_74fHCYWpkJl#/lessons/t-rd1WJbIfOdsFBAgMsJ3749nBemv-Vs

https://view.genially.com/651c3056c1c2ae001106a086